

DKTE'S Textile and Engineering Institute, Ichalkaranji.

Final Year B. Tech. (Semester – VIII)

Artificial Intelligence & Data Science

ADP424 DevOps Lab

Experiment No 1.

Basic Git Operations: Staging, Committing, and Pushing Files to GitHub

Objective

To understand and perform basic **Git version control operations** such as creating files, staging selected files, committing them to the local repository, and pushing the changes to a remote repository on GitHub.

Theory

In **DevOps**, version control systems are essential for managing and tracking changes in code and project files. **Git** is one of the most widely used distributed version control systems. It allows developers to maintain the history of changes, collaborate efficiently, and manage different versions of a project.

Git works with three main areas:

1. **Working Directory** – The local folder where files are created or modified.
2. **Staging Area** – An intermediate area where selected files are prepared before committing.
3. **Repository** – A storage location where committed changes are permanently saved.

In this experiment, a folder is created containing three files: **Code.txt**, **Log.txt**, and **Output.txt**. Only **Code.txt** and **Output.txt** are staged using the `git add` command. These staged files are then committed using the `git commit` command. Finally, the committed files are pushed to a remote repository on **GitHub** using the `git push` command.

This experiment helps understand how developers track and upload project changes in a DevOps workflow.

Keywords

DevOps, Git, GitHub, Repository, Version Control, Staging Area, Commit, Push, Working Directory, Git Commands.

Procedure

1. **Create a new folder** for the experiment.
2. **Create the required files** inside the folder:
 - o Code.txt
 - o Log.txt
 - o Output.txt
3. **Open the terminal/command prompt** and navigate to the folder using:

```
cd folder_name
```

4. **Initialize a Git repository:**

```
git init
```

5. **Add only the required files to the staging area:**

```
git add Code.txt Output.txt
```

6. **Check the status of the files:**

```
git status
```

7. **Commit the staged files** with a commit message:

```
git commit -m "Added Code.txt and Output.txt files"
```

8. **Connect the local repository to the GitHub repository:**

```
git remote add origin <repository_url>
```

9. **Push the committed files to GitHub:**

```
git push origin main
```

10. **Verify the repository on GitHub** to confirm that the files have been successfully uploaded.

Experiment No 2

Branch Management and Merging in Git with Security Patch Update

Objective

To understand how to **update feature branches with the latest changes from the master branch, merge feature branches into master, and push all branches to GitHub using Git commands.**

Theory

In **DevOps and collaborative software development**, Git branches are used to develop features independently without affecting the main codebase. Developers create separate **feature branches** to work on new functionality while the **master (main) branch** contains the stable version of the project.

In this scenario:

- **Feature1 branch** is a **public branch** that can be shared and merged easily.
- **Feature2 branch** is a **private branch** used internally for development.
- The project follows a **monolithic architecture**, meaning the entire application code is contained in a single file.
- A **security patch** was committed to the **master branch**, making both feature branches **one commit behind**.

To maintain consistency, both feature branches must first be **updated with the latest master changes**. This can be done using **merge or rebase** operations. After updating, the features are merged into the master branch.

Finally, the updated branches are **pushed to the remote GitHub repository**, allowing the team to access the updated project.

This process ensures:

- Synchronization between branches
- Integration of new features
- Security updates are included in all branches

Keywords

DevOps, Git, GitHub, Branching, Feature Branch, Master Branch, Merge, Rebase, Commit, Push, Monolithic Architecture, Version Control.

Procedure

Step 1: Fork the Repository

1. Open the given repository on GitHub.
2. Click the **Fork** button to create a copy in your GitHub account.

Step 2: Clone the Forked Repository

Clone the repository to your local system.

```
git clone <repository_url>  
cd repository_name
```

Step 3: Check Available Branches

```
git branch -a
```

This will show branches such as:

- master
- feature1
- feature2

Step 4: Update Feature1 Branch with Security Patch

Switch to the feature1 branch:

```
git checkout feature1
```

Merge the latest changes from master:

```
git merge master
```

Step 5: Update Feature2 Branch with Security Patch

Switch to the feature2 branch:

```
git checkout feature2
```

Merge master into feature2:

```
git merge master
```

Step 6: Apply Feature Branch Changes to Master

Switch to master branch:

```
git checkout master
```

Merge feature1:

```
git merge feature1
```

Merge feature2:

```
git merge feature2
```

Step 7: Push All Branches to GitHub

Push master branch:

```
git push origin master
```

Push feature1 branch:

```
git push origin feature1
```

Push feature2 branch:

```
git push origin feature2
```

Feature branches **feature1** and **feature2** were successfully updated with the security patch from the master branch. The changes from both feature branches were merged into the master branch and all branches were pushed to the GitHub repository successfully.

Experiment No 3

CI Setup Using Jenkins with GitHub and Maven on Ubuntu

Objective

To set up a **Continuous Integration (CI) pipeline using Jenkins**, clone a project from GitHub, push it to a GitHub repository, create a Jenkins Freestyle job, and build the project using Maven.

Theory

Continuous Integration (CI) is a DevOps practice where developers frequently integrate code changes into a shared repository. Each integration is automatically tested and built to detect errors early.

In this experiment, the CI environment is configured on **Ubuntu Linux** with the following tools:

- **Git** – A distributed version control system used to track and manage source code.
- **GitHub** – A remote repository hosting platform used to store and collaborate on code.
- **JDK 11** – Java Development Kit required to compile and run Java applications.
- **Maven** – A build automation and project management tool used for Java projects.
- **Jenkins** – An open-source automation server used to build, test, and deploy software automatically.

The workflow is as follows:

1. A project is cloned from GitHub into the Ubuntu system.
2. Changes or a project are pushed back to the GitHub repository.
3. Jenkins is configured with a **Freestyle job** that connects to the GitHub repository.
4. Jenkins automatically fetches the source code and builds it using Maven.

This process demonstrates a basic **CI pipeline**, which is widely used in DevOps environments.

Keywords

DevOps, Continuous Integration, Jenkins, Git, GitHub, Maven, JDK 11, Ubuntu Linux, Freestyle Job, Build Automation.

Procedure

Step 1: [Verify Required Tools on Ubuntu](#)

Check installed tools:

```
git --version
java -version
mvn -version
jenkins --version
```

Ensure **Jenkins service is running**:

```
sudo systemctl status jenkins
```

Step 2: Clone Repository from GitHub

Clone the repository to the local Ubuntu machine:

```
git clone <repository_url>
cd repository_name
```

Step 3: Push Project to GitHub Repository

Initialize Git (if required):

```
git init
git add .
git commit -m "Initial project commit"
```

Connect to GitHub repository:

```
git remote add origin <github_repository_url>
```

Push the project:

```
git push -u origin main
```

Step 4: Open Jenkins Dashboard

1. Open browser and go to:

```
http://localhost:8080
```

2. Login to Jenkins.

Step 5: Create Jenkins Freestyle Job

1. Click **New Item** in Jenkins dashboard.
2. Enter **Job Name**.

3. Select **Freestyle Project**.
4. Click **OK**.

Step 6: Configure Git Repository in Jenkins

1. Scroll to **Source Code Management**.
2. Select **Git**.
3. Enter the **GitHub Repository URL**.

Example:

```
https://github.com/username/repository_name.git
```

Step 7: Configure Build Step

1. Scroll to **Build Section**.
2. Click **Add Build Step**.
3. Select **Invoke top-level Maven targets**.
4. Enter Maven goals:

```
clean install
```

Step 8: Save and Build the Job

1. Click **Save**.
2. Click **Build Now** to run the job.

Step 9: Check Build Result

1. Go to **Build History**.
2. Click the latest build number.
3. Select **Console Output** to verify build logs.

The GitHub project repository was successfully connected to Jenkins. A Jenkins Freestyle job was created, and the project was built successfully using Maven.

Experiment No 4

Installation of Docker and Implementation of Docker Commands with Deployment to Docker Hub

Objective

To install **Docker on Ubuntu Linux**, perform basic Docker commands, create a **Docker image for an application**, and push the image to **Docker Hub**.

Theory

Docker is an open-source containerization platform used in DevOps to package applications along with their dependencies into lightweight containers. Containers ensure that applications run consistently across different environments such as development, testing, and production.

Docker uses **Docker images** and **containers**:

- **Docker Image** – A read-only template that contains application code, libraries, dependencies, and configuration required to run an application.
- **Docker Container** – A running instance of a Docker image.
- **Docker Hub** – A cloud-based registry service where Docker images can be stored, shared, and downloaded.

In this experiment, Docker is installed on **Ubuntu Linux**. Basic Docker commands are executed to manage images and containers. A Docker image for an application is created using a **Dockerfile**, and the image is then pushed to **Docker Hub** so that it can be accessed and deployed from any system.

This process is widely used in **DevOps pipelines** to simplify application deployment and ensure portability.

Keywords

Docker, Containerization, Docker Image, Docker Container, Dockerfile, Docker Hub, DevOps, Image Repository, Deployment.

Procedure

Step 1: Install Docker on Ubuntu

Update the system packages:

```
sudo apt update
```

Install Docker:

```
sudo apt install docker.io -y
```

Start and enable Docker service:

```
sudo systemctl start docker  
sudo systemctl enable docker
```

Check Docker version:

```
docker --version
```

Step 2: Verify Docker Installation

Run a test container:

```
sudo docker run hello-world
```

This confirms that Docker is installed and working correctly.

Step 3: Basic Docker Commands

List Docker images:

```
docker images
```

List running containers:

```
docker ps
```

List all containers:

```
docker ps -a
```

Pull an image from Docker Hub:

```
docker pull ubuntu
```

Run a container:

```
docker run -it ubuntu /bin/bash
```

Step 4: Create a Dockerfile for the Application

Create a **Dockerfile** in the project directory.

Example Dockerfile:

```
FROM openjdk:11
COPY . /app
WORKDIR /app
RUN javac Hello.java
CMD ["java", "Hello"]
```

Step 5: Build Docker Image

Build the Docker image from the Dockerfile:

```
docker build -t username/myapp .
```

Check created images:

```
docker images
```

Step 6: Run the Docker Container

Run the container from the image:

```
docker run username/myapp
```

Step 7: Login to Docker Hub

Login to Docker Hub from terminal:

```
docker login
```

Enter Docker Hub **username and password**.

Step 8: Push Docker Image to Docker Hub

Tag the image:

```
docker tag username/myapp username/myapp:latest
```

Push the image:

```
docker push username/myapp:latest
```

Experiment No 5

Containerization using Docker and Study of Docker Orchestration Tools – Kubernetes and Docker Swarm

Objective

To understand **containerization using Docker**, create and run containers for an application, and study container orchestration tools such as **Kubernetes and Docker Swarm**.

Theory

Containerization is a lightweight virtualization technique used in DevOps to package applications along with all their dependencies so they can run consistently across different computing environments. Containers isolate the application from the underlying system, ensuring portability and scalability.

Docker is one of the most popular containerization platforms. It allows developers to create **Docker images**, which are templates containing the application and required dependencies. These images can be executed as **containers**.

As applications grow and require multiple containers, managing them manually becomes difficult. This leads to the need for **container orchestration tools**.

Docker Orchestration Tools

1. Kubernetes

Kubernetes is an open-source container orchestration platform used to automate deployment, scaling, and management of containerized applications. It provides features such as:

- Automatic scaling
- Load balancing
- Self-healing containers
- Rolling updates and rollbacks

2. Docker Swarm

Docker Swarm is Docker's native orchestration tool that allows multiple Docker hosts to form a cluster. It helps manage containers across multiple machines and provides:

- High availability
- Load balancing
- Cluster management
- Easy service deployment

In this experiment, Docker is used to containerize an application and basic Docker commands are executed. The concepts and working of orchestration tools such as Kubernetes and Docker Swarm are also studied.

Keywords

Docker, Containerization, Docker Image, Docker Container, Kubernetes, Docker Swarm, Container Orchestration, DevOps, Deployment, Scaling.

Procedure

Step 1: Install Docker

Update system packages:

```
sudo apt update
```

Install Docker:

```
sudo apt install docker.io -y
```

Start Docker service:

```
sudo systemctl start docker  
sudo systemctl enable docker
```

Check Docker installation:

```
docker --version
```

Step 2: Verify Docker Installation

Run a test container:

```
docker run hello-world
```

Step 3: Create an Application File

Example Java program:

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello from Docker Container");  
    }  
}
```

Save as **Hello.java**.

Step 4: Create a Dockerfile

Create a file named **Dockerfile**.

```
FROM openjdk:11
COPY . /app
WORKDIR /app
RUN javac Hello.java
CMD ["java", "Hello"]
```

Step 5: Build Docker Image

```
docker build -t myapp .
```

Check images:

```
docker images
```

Step 6: Run Docker Container

```
docker run myapp
```

This will execute the application inside the container.

Step 7: Study Docker Swarm

Initialize Docker Swarm:

```
docker swarm init
```

Deploy a service:

```
docker service create --name myservice nginx
```

List services:

```
docker service ls
```

Step 8: Study Kubernetes

Kubernetes manages containerized applications through **pods, nodes, and clusters**.

Basic Kubernetes commands:

Check cluster info:

```
kubectl cluster-info
```

List pods:

```
kubectl get pods
```

Deploy application:

```
kubectl create deployment myapp --image=myapp
```

The application was successfully containerized using Docker. Basic Docker commands were executed, and the working concepts of container orchestration tools such as **Kubernetes** and **Docker Swarm** were studied.

Experiment No 6

Implementation of Kubernetes Cluster and Deployment of NGINX Application using NodePort Service

Objective

To deploy a **3-node Kubernetes cluster**, create an **NGINX deployment with multiple replicas**, expose it using a **NodePort service**, and verify the deployment through a web browser.

Theory

Kubernetes is an open-source container orchestration platform used to automate the deployment, scaling, and management of containerized applications. It organizes containers into logical units called **Pods** and manages them across a cluster of machines called **Nodes**.

Key components used in this experiment:

- **Node** – A machine (physical or virtual) that runs containerized applications.
- **Cluster** – A group of nodes managed by Kubernetes.
- **Deployment** – A Kubernetes object that manages the creation and scaling of pods.
- **Replica** – Multiple identical copies of a pod to ensure availability and load balancing.
- **Service** – An abstraction used to expose an application running on a set of pods.
- **NodePort Service** – A service type that exposes an application on a static port of each node's IP address.

In this experiment, a **3-node Kubernetes cluster** is used. An **NGINX deployment with 3 replicas** is created to ensure high availability. A **NodePort service** exposes the deployment so that it can be accessed externally through a browser.

Keywords

Kubernetes, Cluster, Node, Pod, Deployment, Replica, NodePort, NGINX, Container Orchestration, DevOps.

Procedure

Step 1: Create a Kubernetes Cluster with 3 Nodes

Use **Minikube**, **kubeadm**, or a **cloud platform** to create the cluster.

Verify nodes:

```
kubectl get nodes
```

Output should show **3 nodes** in the cluster.

Step 2: Create an NGINX Deployment with 3 Replicas

Run the following command:

```
kubectl create deployment nginx-deployment --image=nginx
```

Scale the deployment to **3 replicas**:

```
kubectl scale deployment nginx-deployment --replicas=3
```

Verify the pods:

```
kubectl get pods
```

Step 3: Verify Deployment

Check deployment details:

```
kubectl get deployments
```

Describe the deployment:

```
kubectl describe deployment nginx-deployment
```

Step 4: Create NodePort Service for NGINX Deployment

Expose the deployment using NodePort:

```
kubectl expose deployment nginx-deployment --type=NodePort --port=80
```

Check the service:

```
kubectl get services
```

Note the **NodePort number** (example: 30007).

Step 5: Access Application from Browser

Open a browser and enter:

```
http://<NodeIP>:<NodePort>
```

Example:

`http://192.168.1.10:30007`

The **NGINX welcome page** should appear, confirming successful deployment.

A **3-node Kubernetes cluster** was successfully deployed. An **NGINX deployment with 3 replicas** was created and exposed using a **NodePort service**, and the application was successfully accessed through a web browser.

Experiment No 7

Scaling Kubernetes Deployment and Creating ClusterIP Services for NGINX

Objective

To modify an existing **Kubernetes deployment**, scale the number of replicas, change the service type to **ClusterIP**, deploy another **NGINX deployment**, and create a **ClusterIP service** for internal communication.

Theory

Kubernetes is a container orchestration system used to manage containerized applications across a cluster of machines. It provides features such as **automatic deployment, scaling, networking, and service discovery**.

Important Kubernetes concepts used in this experiment include:

- **Deployment** – A Kubernetes object that manages pods and ensures the desired number of replicas are running.
- **Replica** – Multiple instances of a pod that improve availability and load balancing.
- **Scaling** – Increasing or decreasing the number of replicas in a deployment.
- **Service** – A stable network endpoint used to access a group of pods.
- **ClusterIP Service** – The default Kubernetes service type that exposes the service only within the cluster.

In this experiment, the previous deployment is modified by increasing the number of replicas from the existing number to **five**. The service type is changed to **ClusterIP**, making it accessible only inside the cluster. Additionally, a new **NGINX deployment with three replicas** is created and exposed through a **ClusterIP service**.

Keywords

Kubernetes, Pod, Deployment, Replica, Scaling, Service, ClusterIP, NGINX, Container Orchestration, DevOps.

Procedure

Step 1: [Use the Previous Deployment](#)

Check the existing deployment:

```
kubectl get deployments
```

Step 2: Change the Replicas to 5

Scale the existing deployment:

```
kubectl scale deployment nginx-deployment --replicas=5
```

Verify the pods:

```
kubectl get pods
```

Step 3: Change Service Type to ClusterIP

Edit the existing service:

```
kubectl edit service nginx-deployment
```

Change the **type** field:

```
type: ClusterIP
```

Verify service type:

```
kubectl get services
```

Step 4: Deploy a New NGINX Deployment with 3 Replicas

Create deployment:

```
kubectl create deployment nginx-app --image=nginx
```

Scale replicas to 3:

```
kubectl scale deployment nginx-app --replicas=3
```

Check pods:

```
kubectl get pods
```

Step 5: Create ClusterIP Service for NGINX Deployment

Expose the deployment internally:

```
kubectl expose deployment nginx-app --type=ClusterIP --port=80
```

Verify the service:

```
kubectl get services
```

Experiment No 8

Deployment of a Web-Based Calculator Application using Kubernetes

Objective

To create a **web-based calculator application**, containerize it using **Docker**, and deploy it on a **Kubernetes cluster** with proper deployment and service configuration.

Theory

A **web-based calculator application** allows users to perform basic arithmetic operations such as addition, subtraction, multiplication, and division through a browser interface.

In **DevOps environments**, applications are often containerized using **Docker** and deployed on container orchestration platforms like **Kubernetes**. Kubernetes manages containerized applications across clusters of machines and ensures scalability, availability, and reliability.

Key Kubernetes components used in this experiment include:

- **Pod** – The smallest deployable unit in Kubernetes that runs one or more containers.
- **Deployment** – Manages pods and ensures the desired number of replicas are running.
- **Replica** – Multiple instances of the same application for load balancing and reliability.
- **Service** – Provides a stable network endpoint to access the deployed application.
- **NodePort Service** – Exposes the application externally so it can be accessed from a browser.

In this experiment, a simple **HTML and JavaScript calculator application** is containerized using Docker and deployed in Kubernetes using a deployment and service.

Keywords

Kubernetes, Docker, Containerization, Deployment, Pod, Replica, NodePort, Web Application, DevOps.

Procedure

Step 1: Create the Calculator Web Application

Create a file **index.html**.

```
<!DOCTYPE html>  
<html>
```

```

<head>
<title>Calculator</title>
<script>
function calculate() {
let a = parseFloat(document.getElementById("num1").value);
let b = parseFloat(document.getElementById("num2").value);
let op = document.getElementById("op").value;
let result;

if(op == "+") result = a + b;
if(op == "-") result = a - b;
if(op == "*") result = a * b;
if(op == "/") result = a / b;

document.getElementById("result").innerHTML = "Result: " + result;
}
</script>
</head>

<body>
<h2>Simple Calculator</h2>

<input type="number" id="num1" placeholder="Number 1">
<input type="number" id="num2" placeholder="Number 2">

<select id="op">
<option value="+>Add</option>
<option value="->Subtract</option>
<option value="*>Multiply</option>
<option value="/>Divide</option>
</select>

<button onclick="calculate()">Calculate</button>

<h3 id="result"></h3>

</body>
</html>

```

Step 2: Create a Dockerfile

Create a **Dockerfile** in the same directory.

```

FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html

```

Step 3: Build Docker Image

```
docker build -t calculator-app .
```

Check the image:

```
docker images
```

Step 4: Push Image to Docker Hub

Login to Docker Hub:

```
docker login
```

Tag the image:

```
docker tag calculator-app username/calculator-app
```

Push the image:

```
docker push username/calculator-app
```

Step 5: Create Kubernetes Deployment

Create a file **deployment.yaml**.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: calculator-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: calculator
  template:
    metadata:
      labels:
        app: calculator
    spec:
      containers:
        - name: calculator
          image: username/calculator-app
          ports:
            - containerPort: 80
```

Apply deployment:

```
kubectl apply -f deployment.yaml
```

Check pods:

```
kubectl get pods
```

Step 6: Create Kubernetes Service

Create **service.yaml**.

```
apiVersion: v1
kind: Service
metadata:
  name: calculator-service
spec:
  type: NodePort
  selector:
    app: calculator
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

Apply the service:

```
kubectl apply -f service.yaml
```

Check service:

```
kubectl get services
```

Step 7: Access the Application

Open a browser and enter:

```
http://<NodeIP>:30007
```

The **web calculator application** will appear and perform calculations.

A **web-based calculator application** was successfully created, containerized using Docker, and deployed on a **Kubernetes cluster** with a deployment and NodePort service, allowing access through a web browser.

Experiment No 9

Installation of Terraform and Performing Basic Terraform Operations

Objective

To install **Terraform** and perform basic infrastructure provisioning operations using Terraform configuration files.

Theory

Terraform is an **Infrastructure as Code (IaC)** tool developed by HashiCorp that allows developers to define and provision infrastructure using configuration files.

Terraform uses a **declarative configuration language called HCL (HashiCorp Configuration Language)** to describe cloud resources. It supports multiple cloud providers such as Amazon Web Services, Azure, and Google Cloud.

Key Terraform commands:

- **terraform init** – Initializes a working directory.
- **terraform plan** – Shows the execution plan.
- **terraform apply** – Creates infrastructure resources.
- **terraform destroy** – Removes the created resources.

Terraform automates infrastructure deployment and ensures **consistency, scalability, and repeatability** in DevOps environments.

Keywords

Terraform, Infrastructure as Code, HCL, DevOps, Cloud Automation, Terraform CLI.

Procedure

Step 1: Install Terraform

Download Terraform from the official website of HashiCorp.

Install using command:

```
sudo apt update
sudo apt install terraform
```

Verify installation:

```
terraform -version
```

Step 2: Create Terraform Configuration File

Create a file **main.tf**

```
provider "aws" {  
    region = "us-east-1"  
}
```

Step 3: Initialize Terraform

```
terraform init
```

Step 4: Check Execution Plan

```
terraform plan
```

Step 5: Apply Terraform Configuration

```
terraform apply
```

Step 6: Destroy Infrastructure

```
terraform destroy
```

Terraform was successfully installed and basic Terraform commands such as **init**, **plan**, **apply**, and **destroy** were executed.

Experiment No 10

Deployment of End-to-End Architecture using Terraform on AWS

Objective

To deploy a complete **cloud infrastructure on AWS using Terraform**.

Theory

Terraform enables automated deployment of cloud infrastructure on platforms such as Amazon Web Services.

Using Terraform, developers can create resources such as:

- Virtual machines
- Networks
- Security groups
- Storage services

The infrastructure configuration is written in **Terraform configuration files (.tf)** and can be reused multiple times.

In this experiment, Terraform is used to create an **AWS EC2 instance** automatically.

Keywords

Terraform, AWS, EC2, Infrastructure as Code, Cloud Deployment, DevOps.

Procedure

Step 1: [Configure AWS CLI](#)

Install and configure AWS CLI:

```
aws configure
```

Enter:

- AWS Access Key
- AWS Secret Key
- Region

Step 2: Create Terraform Configuration

Create **main.tf**

```
provider "aws" {  
    region = "us-east-1"  
}  
  
resource "aws_instance" "example" {  
    ami           = "ami-0abcdef12345"  
    instance_type = "t2.micro"  
  
    tags = {  
        Name = "TerraformInstance"  
    }  
}
```

Step 3: Initialize Terraform

```
terraform init
```

Step 4: Check Execution Plan

```
terraform plan
```

Step 5: Deploy Infrastructure

```
terraform apply
```

Terraform will create an **EC2 instance on Amazon Web Services**.

Step 6: Verify Resources

Check the instance in the AWS Console.

An **end-to-end cloud infrastructure** was successfully deployed on AWS using Terraform.

Experiment No 11

Study and Implementation of AWS DevOps Tools

Objective

To study and implement DevOps services provided by Amazon Web Services such as:

- AWS CodePipeline
- AWS CodeBuild
- AWS CodeDeploy
- AWS CodeStar
- AWS Lambda

Theory

AWS DevOps Tools

1. AWS CodePipeline

AWS CodePipeline is a continuous integration and delivery service used to automate the release pipeline.

2. AWS CodeBuild

AWS CodeBuild compiles source code, runs tests, and produces deployment packages.

3. AWS CodeDeploy

AWS CodeDeploy automates application deployment to servers such as EC2 instances.

4. AWS CodeStar

AWS CodeStar provides a unified interface to manage DevOps projects.

5. AWS Lambda

AWS Lambda is a serverless computing service that runs code without managing servers.

These tools together create a **complete CI/CD pipeline in AWS DevOps environments.**

Keywords

AWS DevOps, CI/CD, CodePipeline, CodeBuild, CodeDeploy, CodeStar, Lambda, Cloud Automation.

Procedure

Step 1: Login to AWS Console

Open the console of Amazon Web Services.

Step 2: Create CodePipeline

1. Open **AWS CodePipeline** service.
2. Create a new pipeline.
3. Connect a source repository (GitHub or CodeCommit).

Step 3: Configure Build Stage

1. Use **AWS CodeBuild**.
2. Define build specifications.

Step 4: Configure Deployment

Use **AWS CodeDeploy** to deploy the application to EC2 instances.

Step 5: Manage DevOps Project

Use **AWS CodeStar** to manage the entire project and pipeline.

Step 6: Run Serverless Function

Create and execute functions using **AWS Lambda**.

DevOps tools provided by **AWS** were studied and implemented to automate **CI/CD pipeline, build, deployment, and serverless execution**.

Experiment No 12

End-to-End Web Application Deployment and CI/CD Pipeline using AWS and DevOps Tools

Objective

To design, implement, and deploy a **complete web application** using **AWS cloud infrastructure** and **DevOps practices**, including:

- Infrastructure provisioning using **Terraform**
- Containerization using **Docker**
- CI/CD pipeline automation using **AWS CodePipeline, CodeBuild, CodeDeploy, CodeStar**
- Orchestration using **Kubernetes**
- Monitoring and serverless functionality using **AWS Lambda**

Theory

Modern software development requires **rapid deployment, scalability, and automation**. This project demonstrates **end-to-end DevOps implementation**:

1. **Infrastructure as Code (IaC)**
Using **Terraform**, the cloud infrastructure is defined and deployed automatically on **AWS**. Resources include EC2 instances, VPC, security groups, S3 buckets, and RDS (if needed).
2. **Containerization**
Application components are containerized using **Docker**. This ensures **consistency** across development, staging, and production environments.
3. **Orchestration**
Kubernetes is used to manage application containers, scale replicas, and ensure high availability.
4. **CI/CD Pipeline**
Using AWS DevOps tools:
 - **AWS CodePipeline** automates the build, test, and deployment workflow.
 - **AWS CodeBuild** compiles the code and runs tests.
 - **AWS CodeDeploy** automates deployment to EC2 or container services.
 - **AWS CodeStar** integrates and manages the DevOps project workflow.
5. **Serverless Functions**
AWS Lambda executes small backend processes or triggers events without managing servers.
6. **Monitoring & Version Control**
GitHub is used for version control, and AWS CloudWatch monitors the deployed application.

The project combines **cloud infrastructure, containerization, orchestration, and automated DevOps pipelines**, giving a complete real-world scenario.

Keywords

AWS, DevOps, CI/CD, Terraform, Docker, Kubernetes, CodePipeline, CodeBuild, CodeDeploy, CodeStar, Lambda, Cloud Automation, Infrastructure as Code, Containerization.

Implementation Procedure

Step 1: Project Setup

- Create a **web application** (e.g., Calculator, To-Do App, or E-Commerce Demo).
- Initialize **Git repository** for version control.

Step 2: Infrastructure Provisioning with Terraform

- Install Terraform on Ubuntu.
- Create **main.tf** to provision AWS infrastructure (VPC, EC2, S3, Security Groups).
- Execute Terraform commands:

```
terraform init
terraform plan
terraform apply
```

Step 3: Containerize Application using Docker

- Create **Dockerfile** for the web application.
- Build Docker image:

```
docker build -t myapp .
```

- Test locally and push to **Docker Hub**:

```
docker push username/myapp
```

Step 4: Kubernetes Deployment

- Deploy Dockerized app on **Kubernetes cluster**.

- Create **deployment.yaml** and **service.yaml** (NodePort/ClusterIP).
- Apply manifests:

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

- Verify application using browser:

`http://<NodeIP>:<NodePort>`

Step 5: CI/CD Pipeline using AWS DevOps Tools

1. **AWS CodePipeline** – Automates source, build, and deployment.
2. **AWS CodeBuild** – Compiles code and builds Docker images.
3. **AWS CodeDeploy** – Deploys Docker containers to EC2 or EKS.
4. **AWS CodeStar** – Manages the entire DevOps project and workflow.
5. Configure **GitHub repository** as source.

Step 6: Serverless Integration

- Create **AWS Lambda function** for event-driven backend tasks.
- Integrate Lambda with S3, DynamoDB, or CloudWatch as triggers.

Step 7: Testing and Monitoring

- Test the application functionality across multiple replicas.
- Use **AWS CloudWatch** to monitor logs, metrics, and alarms.
- Verify automated deployment through pipeline changes in GitHub.

Result

- Successfully deployed a **web application** on AWS with **Kubernetes orchestration**.
- Application is **containerized** with Docker and scalable using Kubernetes.
- Implemented **CI/CD pipeline** using AWS DevOps tools for automated deployment.
- Monitored application health and executed **serverless functions** using AWS Lambda.
- Demonstrated **end-to-end DevOps workflow** from code commit to production deployment.

The Capstone Project successfully demonstrates **real-world DevOps practices** using AWS cloud infrastructure, Terraform, Docker, Kubernetes, and AWS DevOps tools. This end-to-end

implementation highlights the importance of **automation, scalability, monitoring, and continuous deployment** in modern software development.